МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Построение и анализ алгоритмов»

Тема: «Кнут-Моррис-Пратт»

Студент гр. 4343 _____ Толмачев И.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Нахождение всех вхождений P в T

Задание.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P (| P| ≤25000) и текста T (| T| ≤5000000) найдите все вхождения P в T.

Вход:

- Первая строка — P

- Вторая строка — T

Выход:

индексы начал вхождений P в T, разделённые запятой; если P не входит в T, то вывести -1.

Описание алгоритма и оценка его сложности.

Алгоритм Кнута-Морриса-Пратта решает задачу поиска всех вхождений образца *P* в текст *T*. Его ключевая особенность — предварительная обработка образца за линейное время, которая в дальнейшем позволяет выполнять поиск без возвратов по тексту.

Построение префикс функции. На первом этапе для образца *P* строится массив *prefix* длины *m*. Элемент *prefix*[*i*] равен длине наибольшего собственного префикса подстроки *P*[0…*i*], который одновременно является её суффиксом. Иными словами, *prefix*[*i*] указывает, сколько начальных символов образца совпадает с окончанием фрагмента, заканчивающегося в позиции *i*.

Вычисление происходит итеративно: *prefix*[0] всегда равен 0. Для каждого индекса *i* от 1 до *m*-1: запоминается текущая длина совпадающего префикс суффикса *j* = *prefix*[*i*-1]; пока *j* > 0 и *P*[*i*] != *P*[*j*], выполняется откат: *j* = *prefix*[*j*-1]. Это позволяет найти более короткий префикс, который может совпасть с окончанием, включающим *P*[*i*]; если после откатов символы *P*[*i*] и *P*[*j*] равны, *j* увеличивается на 1; *prefix*[*i*] присваивается значение *j*.

Поиск вхождений. После построения *prefix* выполняется однопроходное сканирование текста *T*. Поддерживаются два указателя: *i* —

позиция в тексте; j — позиция в образце. Для каждого символа $T[i]$ производятся следующие действия: пока $j > 0$ и $T[i]$!= $P[j]$, индекс j откатывается по правилу j = *prefix*[j-1]. Это эквивалентно сдвигу образца вперёд на j - *prefix*[j-1] позиций без возврата i; если $T[i]$ == $P[j]$, j увеличивается на 1; когда j становится равным m, это означает, что найдено вхождение, начинающееся в позиции $i - m + 1$. Индекс начала добавляется в список positions, после чего для продолжения поиска j переустанавливается в *prefix*[m-1].

Временная сложность: O($m + n$), где m — длина образца, n — длина текста. Этап предобработки выполняется за O(m), а последующий поиск всех вхождений за O(n). Оба этапа линейны относительно длины своих входных данных, а суммарное время линейно зависит от общего объёма исходных строк.

Пространственная память: в представленной реализации алгоритма используется дополнительная память для хранения массива *prefix* размером O(m) и вектора *positions*, который в худшем случае сохраняет n индексов, занимая O(n) памяти. Таким образом, общий объём дополнительной памяти в худшем случае составляет O($m + n$).

Тестирование.

Для проверки корректности алгоритма Кнута-Морриса-Пратта было проведено автоматическое тестирование с использованием фреймворка Google Test. Тесты охватывают как типичные ситуации, так и граничные случаи.

EmptyPattern - проверяет случай пустого образца: P = "", T = "abc". Алгоритм сразу возвращает «-1», так как поиск пустой строки не осуществляется.

PatternNotFound - образец отсутствует в тексте: P = "abc", T = "defgh". Возвращается «-1».

SingleMatch - одно вхождение образца: P = "abc", T = "xyzabc123".

Индекс начала вхождения равен 3, результат «3».

MultipleMatches - несколько перекрывающихся вхождений: *P* = "aa", *T* = "aaa". Ожидаются индексы 0 и 1, результат «0,1».

PatternEqualsText - образец полностью совпадает с текстом: *P* = "abc", *T* = "abc". Возвращается «0».

PatternLongerThanText - образец длиннее текста: *P* = "abcdef", *T* = "abc". Вхождение невозможно, результат «-1».

OverlappingMatches - перекрывающиеся вхождения с более сложным образцом: *P* = "aba", *T* = "ababa". Вхождения начинаются с индексов 0 и 2, результат «0,2».

Все тесты завершились успешно, подтвердив корректность работы алгоритма для заданных условий.

Определить, является ли A циклическим сдвигом B

Задание.

Заданы две строки A (|A| ≤5000000) и B (|B| ≤5000000).

Определить, является ли A циклическим сдвигом B (это значит, что

A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A

- Вторая строка — B

Выход:

Если A является циклическим сдвигом B, то индекс начала строки B в A; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Описание алгоритма и оценка его сложности.

Основная идея: *A* является циклическим сдвигом B тогда и только тогда, когда *B* входит как подстрока в строку *A*+*A*. При этом искомый индекс начала *B* в *A* равен начальной позиции первого вхождения *B* в *A*+*A*, которая меньше *n*.

Построение префикс функции для строки *B*. Префикс функция *prefix* длины *m* определяется так: *prefix*[*i*] — длина наибольшего собственного префикса подстроки *B*[0..*i*], который одновременно является её суффиксом. Вычисляется итеративно: *prefix*[0] = 0; Для *i* = 1…*m*-1: *j* = *prefix*[*i*-1]; пока *j* > 0 и *B*[*i*] != *B*[*j*], *j* = *prefix*[*j*-1]; если *B*[*i*] = *B*[*j*], *j* = *j*+1. *prefix*[*i*] = *j*.

Поиск образца *B* в виртуальной строке *A*+*A*. Фактически строка *A*+*A* не строится, а вместо этого используется циклический доступ к символам A: для индекса *i* в диапазоне [0, 2*n*-1] берётся символ *cur* = *A*[*i mod n*].

Поиск ведётся классическим алгоритмом КМП: инициализируется длина совпадения *j* = 0; Для *i* = 0…2*n*-1: пока *j* > 0 и *cur* !=*B*[*j*], выполняется откат *j* = *prefix*[*j*-1]; если *cur* = *B*[*j*], *j* = *j*+1; если *j* = *m* найдено полное совпадение: вычисляется начало вхождения в *A*+*A*: *start* = *i* - *m* + 1; если *start* < *n*, то это

первое корректное вхождение, и ответ — *start*. Алгоритм завершается; иначе вхождение начинается во второй копии A, поиск продолжается с $j = prefix[j-1]$.

Временная сложность: построение префикс функции требует $O(m)$ операций. Поиск по виртуальной строке $A+A$ выполняет не более $2n$ итераций, каждая из которых амортизированно занимает $O(1)$ из-за откатов по *prefix*. Суммарное время — $O(n+m)$.

Пространственная сложность: хранятся исходные строки A и B — $O(n+m)$ и массив префикс функции prefix длины $O(m)$. Виртуальная удвоенная строка не создаётся. Дополнительные переменные: индексы, счётчики, используют $O(1)$. Общий расход памяти — $O(n+m)$.

Тестирование.

Для проверки корректности алгоритма определени циклического сдвига было проведено автоматическое тестирование с использованием фреймворка Google Test. Тесты охватывают как типичные ситуации, так и граничные случаи.

DifferentLength - строки разной длины: A = "abc", B = "ab". Алгоритм сразу возвращает -1, поскольку циклический сдвиг возможен только при равных длинах.

EmptyStrings - обе строки пусты: A = "", B = "". Пустая строка считается циклическим сдвигом самой себя, индекс начала 0.

IdenticalStrings - идентичные непустые строки: A = "abc", B = "abc". Ожидается индекс 0.

NormalShift - стандартный циклический сдвиг: A = "defabc", B = "abcdef". Сдвиг на 3 символа, индекс начала B в A равен 3.

NotAShift - строки одинаковой длины, но не являются сдвигами: A = "abc", B = "abd". Возвращается -1.

AllSameCharacters - строки состоят из одного повторяющегося символа: A = "aaaa", B = "aaaa". Индекс начала 0, так как любое положение подходит, но алгоритм находит первое вхождение.

ShiftByOne - сдвиг на один символ: *A* = "ab", *B* = "ba". Индекс начала *B* в *A* равен 1.

ShiftBySeveral - сдвиг на несколько символов в строке длиной 5: *A* = "eabcd", *B* = "abcde". Первое вхождение *B* в *A*+*A* начинается с индекса 1, ответ 1.

Все тесты завершились успешно, подтверждая корректность работы алгоритма.